Algoritmos de búsqueda en árboles

Arturo Gil y Juan José Cabrera

27 de octubre de 2026

1. Introducción

En esta práctica utilizaremos y programaremos diferentes algoritmos de búsqueda en árbol. Durante las clases magistrales de la asignatura hemos planteado diferentes tipos de problemas y soluciones que se pueden relacionar con lo que veremos en esta sesión, por ejemplo:

- Problemas de tipo 8-puzzle.
- Hallar el trayecto con menor cantidad de transbordos entre dos estaciones de metro o autobús.
- Búsqueda del camino más corto entre dos ciudades.

2. El problema

Nos centraremos en un problema de búsqueda de rutas entre dos ciudades. Trabajaremos inicialmente con el mapa mostrado en la figura 1. Se trata de un mapa de carreteras de España simplificado. Los ejes X e Y de la gráfica (grados) muestran la posición GPS de algunas ciudades españolas (nodos de la red). Entre los nodos de la red **se han anotado la distancia en kilómetros cuando se circula por carretera entre ellos**. Durante el resto de la práctica, abordaremos los siguientes objetivos:

- Ser capaces de representar grafos de forma sencilla en python.
- El manejo de listas y diccionarios en python para problemas de búsqueda en árboles.
- La programación de los algoritmos siguientes para la búsqueda de rutas óptimas entre ciudades
  - Breadth-first search BFS: búsqueda en anchura.
  - Búsqueda greedy (ávara) y
  - A^* (el algoritmo A estrella).

2.1. Materiales

Utilizaremos el repo Intro2AI: `https://github.com/4rtur1t0/Intro2AI`. Usaremos los ficheros del directorio `practice_search_algorithms`. Dentro de este directorio, encontramos:

- `search_algorithms.py`: fichero principal de la práctica. Desde aquí llamaremos al resto de algoritmos que vamos a programar y a probar.
- `graph/graph.py`: una clase `Graph` con una implementación sencilla de un grafo no dirigido en python.
- `search_algorithms`: un paquete de python donde el estudiante deberá implementar sus algoritmos de búsqueda en árboles para después poderlos probar.

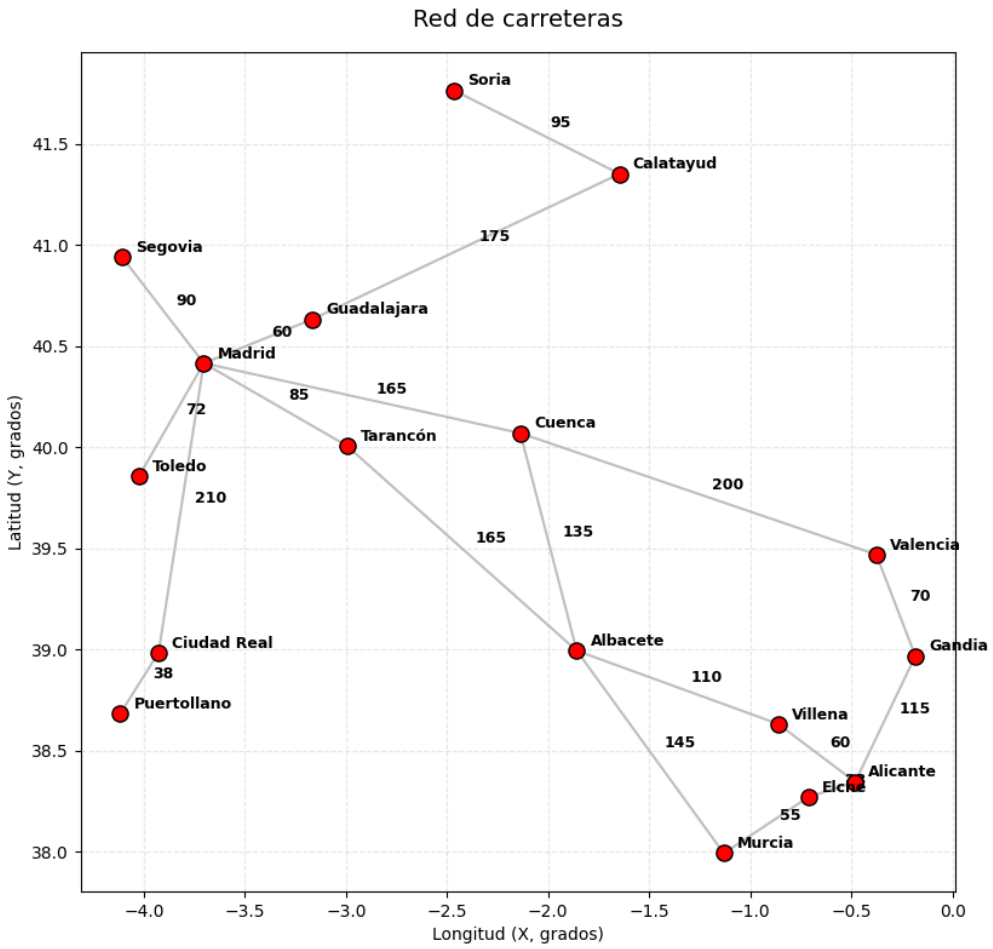


Figura 1: Un mapa de carreteras de España simplificado. Se muestra la distancia por carretera entre los diferentes nodos y la posición GPS (grados) de cada nodo.

2.2. ¿Cómo representar un árbol en python?

Como ayuda, la clase `Graph` se proporciona ya programada, pues queremos que el/la estudiante se concentre en implementar los algoritmos de búsqueda sin tener que crear toda la estructura del programa desde cero. También, se han proporcionado una serie de funciones al estudiante que le facilitarán el trabajo, por ejemplo: una función para dibujar el grafo, encontrar un nodo en el grafo, encontrar la distancia entre dos nodos o calcular la distancia en línea recta hasta un nodo.

Ejercicio 2.1

En este momento es importante que entienda cómo se ha programado el grafo.

EJERCICIO
Observe el resultado de la línea `print(spain_network.graph)` del script `search_algorithms.py`. Alternativamente, observe la variable `print(spain_network.graph)` en el depurador (figura 4).

Veamos, a continuación, algunos ejemplos en el diccionario `spain_network.graph` (clase `Graph()`):

```
{'Madrid': {'Ciudad Real': 210, 'Cuenca': 165, 'Guadalajara': 60,
            'Segovia': 90, 'Tarancón': 85, 'Toledo': 72},
 'Cuenca': {'Albacete': 135, 'Madrid': 165, 'Valencia': 200},
 'Albacete': {'Cuenca': 135, 'Murcia': 145, 'Tarancón': 165, 'Villena': 110}, ...}
```

Este diccionario está indicando que Madrid está conectado con Ciudad Real, Cuenca, Guadalajara, Segovia, Tarancón y Toledo. Cada uno de los valores en el diccionario guarda la distancia (en kilómetros) por carretera. A su vez Cuenca está conectada con Albacete, Madrid y Valencia. Para encontrar todos los nodos que están conectados con Cuenca, hacemos (accedemos a la clave `'Cuenca'`):

```
print(spain\_network.graph['Cuenca'])
```

Nos debemos fijar que la variable `spain_network.graph` es un diccionario de diccionarios (pues en cada par *key-value*, *value* es un diccionario también). Para ver la distancia entre Cuenca y Valencia, accedemos a:

```
print(spain\_network.graph['Cuenca']['Valencia'])
```

Obviamente, debemos obtener el mismo resultado si buscamos la distancia entre Valencia y Cuenca:

```
print(spain\_network.graph['Valencia']['Cuenca'])
```

Esta no es la forma más eficiente de guardar un grafo, pero sí la más legible y fácil de entender (por esta razón se ha elegido esta representación en la práctica). La clase `Graph()` cuenta con varios métodos que nos permiten manejar el grafo con facilidad:

```
def get_node(self, name):
def get_distance(self, node1, node2):
def get_neighbors(self, node):
def compute_flying_distance(self, node1, node2):
```

Ejercicio 2.2

Con esto, abra el fichero `search_algorithms.py`. Verá las siguientes líneas de ćodigo:

```
if __name__ == "__main__":
    spain_network = Graph()
    spain_network.build_network()
    print(spain_network.graph)
    spain_network.plot_network()
```

EJERCICIO
Utilice las siguientes funciones sobre el objeto `spain_network` y anote el resultado: ¿qué hace cada función? ¿Qué resultado se obtiene?

```
get_node(self, name)
get_distance(self, node1, node2)
def get_neighbors(self, name)
def compute_flying_distance(self, ciudad1, ciudad2)
```

Puede consultar la implementación de cada función en la clase `graph/Graph()`.

2.3. Búsqueda en anchura (*breadth-first search*)

Un algoritmo de búsqueda en anchura o Breadh-first Search (BFS) explora el árbol según el pseudo-código mostrado en el Algoritmo 1. El algoritmo explora en un orden fijo todos los nodos de la red hasta encontrar una solución. Los nodos se exploran por niveles utilizando una cola FIFO (*First In, First Out*). Los niveles se definen en base a los saltos que hay que dar para conectar el nodo inicial (*root*) con cualquier otro nodo. La característica principal del algoritmo BFS consiste en que se asegura la exploración de todos los nodos de un nivel antes de pasar al siguiente nivel. Este forma de explorar el árbol se muestra en la figura 2, donde se describe también el contenido de la cola abierta FIFO (*open queue*) que se emplea para almacenar los nodos que se pretenden visitar.

Para nosotros, la cola abierta será simplemente una lista de python con los nombres de los nodos, donde añadiremos elementos por la derecha (`append`) y los eliminaremos por la izquierda (`pop(0)`). Un ejemplo de cómo se procesa el árbol lo vemos en la figura 3. El funcionamiento del algoritmo 1 lo podemos describir de la manera siguiente:

- Se inicializa la cola abierta del algoritmo (cola FIFO, *open queue*) con el nodo inicial (este es el nodo inicial desde el que queremos encontrar el camino).
- Se inicializa la lista cerrada (`self.closed_queue`). Inicialmente, esta lista estará vacía. Después, contendrá los nodos que ya han sido explorados.
- Se inicializa un diccionario (`node_info`) que contiene información sobre el padre de cada nodo (*id est*, desde qué nodo se ha llegado a cada nodo). El nodo inicial se marca con `None`, pues no tiene padre.
- El algoritmo prosigue, indicando que mientras haya elementos dentro de la cola abierta:
  - Se extrae un elemento por la izquierda de la cola abierta (`queue.pop()`).
  - Si el nodo actual es igual al nodo buscado (`destination_node`), se termina la búsqueda y se calcula la ruta.
  - En caso contrario, se procesa el nodo actual. El procesado del nodo actual se debe hacer en la función (`process_neighbours`). Dentro de esta función:
    - Se buscan los nodos vecinos que están conectados al nodo actual (`neighbours`). Para cada nodo vecino:
      - Si el nodo está en la lista cerrada: `continue`, pues ya ha sido explorado anteriormente.

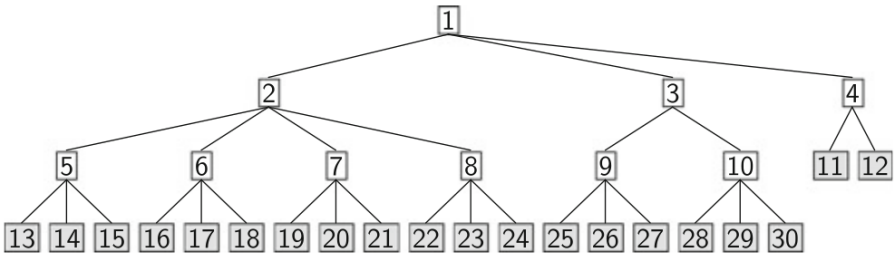
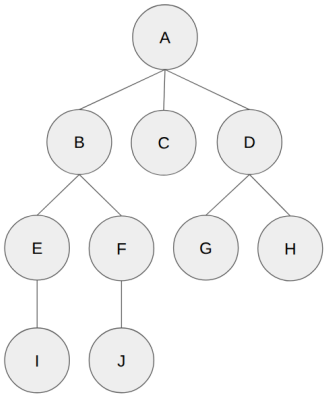


Figura 2: Un árbol y el orden en el que se exploran los nodos en *breadth-first search*.



ITERACIÓN	POP (izquierda)	APPEND (derecha)	COLA
0	-	A	A
1	A	B, C, D	B C D
2	B	E, F	C D E F
3	C	-	D E F
4	D	G, H	E F G H
5	E	I	F G H I
6	F	J	G H I J
7	G	-	H I J
8	H	-	I J
9	I	-	J
10	J	-	-

Figura 3: Izquierda: un árbol. Derecha: el orden en el que se exploran los nodos, las operaciones de *pop* y *append*y la cola del algoritmo *breadth-first search* en cada una de sus iteraciones.

- ◇ Así pues, si el nodo vecino no está en la lista cerrada, se añadirá a la lista abierta (si no existe en la lista abierta). En este mismo momento, se añadirá el nodo vecino y su padre en el diccionario `self.parent_info`

Ejercicio 2.3

Continuamos la práctica y seguimos leyendo el fichero `search/search_algorithms.py`. Descomente las siguientes líneas de ćodigo:

```
algoritmo = BFS_Algorithm(spain_network)
origen = 'Madrid'
destino = 'Elx'
route, distance, iterations = algoritmo.find_route(origen, destino)
```

EJERCICIO

- Encuentre la clase `BFS_Algorithm`. Esta clase la encontrarás en el fichero de python `search_algorithms/bfs_algorithm.py`. La clase implementa el algoritmo BFS que se ha explicado.
- Encuentre la función `find_route` deberá entender las líneas del algoritmo y relacionarlas con el algoritmo BFS que ya se ha expuesto.



Figura 4: Un diccionario en python representando un grafo.

Data:
start = root_node
end = end_node
open_queue = [root_node]
closed_queue = []
node_info = {root_node: None}
Result: result = $[node_x, node_y, node_z, \ldots]$
while *len(queue)*> 0 **do**
    node = open_queue.pop(0)
    closed_queue.append(node)
    if *node == end_node* **then**
        route = compute_route(node_info);
        return route;
    else
        /* Se añaden los vecinos del nodo actual a la cola*/
        process_neighbors(node)
    end
end

Algorithm 1: El algoritmo BFS.

Data: current_node
Result: None
neighbours = get_neighbours(current_node)
for *(neighbour in neighbours)* **do**
    if *neighbour in closed_queue* **then**
        continue
    end
    if *neigbour not in visited_nodes* **then**
        add neighbour to the open queue
        store parent information
    end
end

Algorithm 2: Process neighbours.

Ejercicio 2.4

Deseamos entender cómo evoluciona la cola abierta (`self.open_queue`) y la cola cerrada `self.closed_queue` del algoritmo en cada iteración. También es interesante comprender cómo funciona la variable `self.node_info` (información de los nodos) y la lista de vecinos en la función `process_neighbors`.

EJERCICIO

a) Modifique la función `find_route` para que imprima por pantalla estas 4 variables. Se recomienta que escriba una función `print_info` que se encargue de imprimir esta información.

b) Ejecute la función `find_route` paso por paso usando el modo debug. Anote en papel una tabla como la mostrada en la figura 3. Anote el contenido de la lista cerrada y abierta en las 3 primeras iteraciones.

c) Siga el progreso de las listas abierta, cerrada y los vecinos en el mapa de la figura 1.

Es fácil ver cómo se van añadiendo los nodos a la cola abierta (`self.open_queue`) y cerrada (`self.closed_queue`). Nos tenemos que dar cuenta de que los nodos en la cola se procesan por la izquierda. Los nuevos nodos que se encuentran conectados al nodo actual se añaden por la derecha. Con esta acción, nos aseguramos de que todos los nodos conectados a un nivel determinado hayan sido procesados antes de avanzar hacia un nivel más bajo. En la práctica nos debemos dar cuenta de que el algoritmo busca "en anillos" expandiéndose desde el nodo inicial hasta dar con el nodo final.

2.3.1. Detalles importantes

Hay un detalle importante que hemos soslayado: el mapa mostrado es un tipo especial de árbol en el que un nodo hijo puede estar conectado con varios padres (podemos llegar a Albacete desde Cuenca o Tarancón). Para guardar la ruta final en un algoritmo BFS en el caso en el que un nodo pueda ser visitado desde diferentes padres, la mejor estrategia es utilizar un diccionario de padres (`node_info`): como el algoritmo BFS explora el grafo por niveles (saltos), la primera vez que se visita cualquier nodo se tiene la garantía de que se ha llegado a él con el mínimo número de saltos. Así pues, si, posteriormente, se llega al mismo nodo por otro camino (desde otro padre), el número de saltos será igual o mayor que en el primer caso. En consecuencia, solamente se debe añadir el nodo a la cola la primera vez que se vea. Esto se puede observar en el algoritmo BFS que se ha programado, pues solamente se añade un nodo al diccionario `node_info` si no está en `closed_queue` y tampoco está en `open_queue`.

Ejercicio 2.5

Es importante ver cómo se reconstruye el camino que constituye la solución. Cuando, finalmente, el nodo actual coincide con el nodo de destino, se llama a la función `reconstruct_route`.

EJERCICIO
Programe la función `reconstruct_route`. La función debe consultar el diccionario `parent_info` y construir la ruta desde el final hasta el principio. Como ayuda, imprima por pantalla el nodo actual y su padre mientras se construye la ruta. Calcule la distancia total de la ruta para que sea devuelta por el algoritmo.

2.3.2. Pruebas finales

Para finalizar, haremos una serie de pruebas para poder observar las características más importantes del algoritmo. En un algoritmo de búsqueda en grafos, nos interesa saber, principalmente:

- ¿Cuántas iteraciones son necesarias para encontrar una solución entre dos nodos del grafo?

- ¿Cuántos nodos se han tenido que expandir para encontrar una solución?

Ambos factores determinan el tiempo de cómputo del algoritmo. El segundo factor está relacionado con la cantidad de memoria que necesita el algoritmo para encontrar una solución. En este ejemplo tenemos muy pocos nodos: el tiempo de cómputo y la necesidad de memoria son factores importantes en problemas búsqueda reales, como el que veremos al final de esta práctica.

Ejercicio 2.6

Se desea calcular el número de iteraciones realizadas y el tamaño máximo de las colas (abierta y cerrada). Esto nos dará una idea del consumo en tiempo y memoria del algoritmo.

EJERCICIO
Modifique el código de la función `find_route` para contar el número de iteraciones del algoritmo y el número de nodos que se han añadido a la cola.

Ejercicio 2.7

Necesitamos tener una batería de pruebas de nuestro algoritmo. Probaremos con un conjunto de nodos de origen y destino.

EJERCICIO
Modifique los puntos de origen y destino en el fichero `search_algorithms.py`. Después de 10 pruebas diferentes,

- Compruebe si en todos los casos se encuentra una solución.
- Anote la ruta obtenida, la distancia, el número de iteraciones del algoritmo y el número de nodos expandidos.
- Suponga que se ha encontrado una ruta entre dos ciudades del mapa. ¿La distancia total por carretera entre las dos ciudades es mínima?
- ¿El número de saltos entre nodos consecutivos de la ruta entre las ciudades es mínimo?

Después de haber jugado un poco con el algoritmo y haber buscado rutas entre diferentes nodos del mapa, nos debemos dar cuenta de que:

- Si existe una ruta entre dos nodos del mapa, la ruta encontrada por BFS es aquella con el menor número de saltos entre nodos.
- De ninguna manera, la ruta hallada es mínima en términos de distancia: es mínima en términos del número de saltos entre nodos.

Por ejemplo, si buscamos una ruta entre Madrid y Elx, obtendremos:

```
Found destination! In iterations:  17
Route found: Madrid -> Cuenca -> Albacete -> Murcia -> Elx
TOTAL DISTANCE IS:  500
```

Así pues, el algoritmo necesita 17 iteraciones para encontrar una ruta entre las dos ciudades. La ruta obtenida tiene un número de saltos mínimo (4, 4 flechas). La distancia obtenida (500 km) no es la mínima entre los nodos (pues el algoritmo no usa ninguna información de distancia).

2.3.3. Aplicaciones

Podríamos preguntarnos para qué aplicaciones nos interesaría saber la distancia entre dos nodos en base al número de saltos. Se anotan algunas situaciones:

- **Protocolos de enrutamiento de red**: en el algoritmo RIP (*Routing Information Protocol*), los routers utilizan la cuenta de saltos (*hop count*) para encontrar el camino que necesita el menor número de *routers* intermedios para enviar los datos entre dos puntos de internet. Esta es una cuestión importante, pues en un *router* se produce siempre un retardo en el mensaje (al tener que procesar el mensaje y enrutarlo a otra red). La distancia total entre el nodo inicial y el final es menos relevante que el número de nodos intermedios: piense que entre dos nodos la transmisión de información se puede hacer usando fibra óptica y donde los datos viajan a la velocidad de la luz (en ese medio).

- **Algoritmos para la transmisión de datos en redes radio de tipo *mesh***: en las redes de radio de tipo *mesh* (por ejemplo, cuando se usan sistemas de radio XBee), cada elemento de la red está conectado con otros elementos de tipo radio. En este tipo de redes se precisa que haya el menor de saltos entre el emisor y el receptor del mensaje, mientras que se limita el número de veces que hay que retransmitir el mensaje.

2.4. *Greedy search*

El algoritmo BFS que hemos visto se clasifica como un algoritmo de búsqueda no informado, pues no dispone de ninguna información que le indique si se está acercando al nodo que se está buscando o si se encuentra cerca de encontrar una ruta válida hasta él.

Seguidamente, vamos a partir del algoritmo BFS y lo vamos a modificar utilizando una información de distancia. En concreto, vamos a utilizar un función heurística $h(s)$ que llamaremos "distancia en línea recta" o "*flying distance*". En vez de expandir los nodos de la cola en un orden fijo, vamos a expandir primero el nodo de la cola que tenga un valor de $h(s)$ menor. Esta forma de proceder se denomina, generalmente, avara (Greedy), de ahí que este algoritmo se denomine *Greedy Best-First Search*.

En este algoritmo avaro (*greedy*), se calcula la función $h(s)$ para cada nodo de la cola abierta (`self.open_queue`) y, a continuación, se reordena la cola en base a esta distancia. Con esto, los nodos con distancia $h(s)$ menor se quedarán a la izquierda de la cola y serán procesados antes. Con esto, en resumen, nos damos cuenta de que:

- **El algoritmo BFS es de tipo no informado**, pues no usa información sobre la distancia entre los nodos o la distancia de un nodo al destino.

- **El algoritmo Greedy es de tipo informado** y trata de centrarse en los nodos para los que la distancia en línea recta es menor.

En afegir una mètrica greedy basada en la distància en línia recta (flying distance) cap al destí, l'algoritme passa de ser un BFS pur a convertir-se en un Greedy Best-First Search. La cua FIFO tradicional es reemplaça per una cua que prioritza els nodes més propers al destí final. Les dues característiques principals que s'hi troben són:

- **Rapidesa d'exploració**: l'algoritme "avança" directament en la direcció geogràfica del destí, reduint dràsticament el nombre de nodes explorats en comparació amb l'exploració en anells del BFS.

- **Pèrdua de la garantia d'optimitat**: a diferència del BFS estàndard, el Greedy Best-First Search no garanteix trobar el camí amb el mínim nombre de salts si hi ha obstacles, tancament de carreteres o connexions indirectes.

La clase `Graph()` proporciona una función para calcular esta distancia en línea recta. La distancia (en kilómetros) se calcula en base a la posición GPS de cada ciudad (que convenientemente se ha almacenado en un array). Para el cálculo de esta distancia se utiliza la función "haversine" o semiverseno que devuelve la distancia más corta entre dos puntos de una esfera:

```
self.graph.compute_flying_distance(node_name, destination_name)
```

Ejercicio 2.8

Se propone al estudiante que programe el algoritmo Greedy BFS partiendo del algoritmo BFS.

EJERCICIO
Debe implementar el algoritmo avaro (*greedy*). Para ello:

- Parta del fichero `search_algorithms/greedy_algorithm.py` y modifíquelo.
- Modifique la función `find_route` y `process_neighbours`. Instrucciones:
  - En la función `process_neighbors`, al añadir un nodo a la cola abierta, se deberá calcular la distancia en línea recta hasta el nodo destino (*flying distance*).
  - Guarde esta información en el diccionario `self.h`.
  - Reordene la cola abierta en base a las distancias de cada nodo al destino.

Con todo lo anterior, descomente las siguientes líneas (y comente las líneas anteriores):

```
algoritmo = Greedy_Algorithm(spain_network)
origen = 'Madrid'
destino = 'Elx'
route, distance, iterations = algoritmo.find_route(origen , destino)
```

Ejercicio 2.9

Deseamos realizar varias pruebas con el nuevo algoritmo avaro que acabamos de programar.

EJERCICIO
Pruebe a calcular la ruta entre Madrid y Elx. A continuación, pruebe a calcular la ruta entre Elx y Madrid. ¿Se obtiene el la misma ruta (a la inversa)?¿Se obtiene la misma distancia total? ¿Considera que la ruta calculada es óptima?

Ejercicio 2.10

Deseamos realizar varias pruebas con el nuevo algoritmo avaro que acabamos de programar.

EJERCICIO
Pruebe en total 5 rutas (p. ej. Madrid-Elx, Puertollano-Murcia, Gandía-Segovia. Para cada ruta, prueba BFS y pruebe el algoritmo Avaro:

- Anote la ruta calculada.
- Anote el número de iteraciones del algoritmo.
- Anote el número de nodos expandidos por el algoritmo.
- Anote la distancia total de la ruta calculada. ¿La ruta hallada es óptima?

Nos debemos dar cuenta de que este algoritmo *greedy* es más eficiente que el anterior en términos de iteraciones, pues siempre intenta expandir el nodo con menor distancia en línea recta. De esta manera, se dirige la búsqueda de forma más rápida hacia el destino deseado. También, el número total de nodos que se expanden es menor (si lo comparamos con BFS). Vemos, sin embargo, que la ruta que encontramos en el caso de Madrid-Elche no es óptima.

Ejercicio 2.11

Deseamos saber en qué punto se equivoca el algoritmo *Greedy* y por qué razón.

EJERCICIO
Calcule a mano la ruta óptima entre Madrid y Elx: es fácil, no hay muchas combinaciones posibles. Después:

- Pruebe de nuevo a hallar una ruta entre Madrid y Elx usando el algoritmo avaro.
- Ejecute el algoritmo paso a paso, observando los nodos y sus *flying distances*.
- ¿En qué momento se equivoca el algoritmo avaro? ¿Por qué la ruta que se calcula no es óptima?

Si buscamos la ruta Madrid-Elx con el algoritmo Greedy, obtenemos:

```
Found destination! in iterations: 6
Route found: Madrid -> Cuenca -> Albacete -> Villena -> Alicante -> Elche
TOTAL DISTANCE IS: 493
```

Vemos que el algoritmo necesita 6 iteraciones para encontrar una ruta entre las dos ciudades. Sin duda, utilizar $h(s)$ como heurístico permite centrar la búsqueda en los nodos más prometedores (que están más cerca del nodo de destino, en línea recta). La ruta entre nodos tiene 5 saltos (5 flechas), 1 más que el algoritmo BFS. La ruta obtenida entre las dos ciudades no es la que tiene una distancia mínima. En concreto, en la primera iteración, el algoritmo decide, de forma equivocada, viajar a Cuenca en vez de viajar a Tarancón. Esta decisión la toma porque Cuenca tiene una *flying distance* (hasta Elche) menor que Tarancón. A partir de esa equivocación el algoritmo sigue avanzando de forma ávida hacia el destino y, además, no es capaz de enmendar su error.

2.4.1. Detalles de implementación

En esta práctica hemos mantenido una lista ordenada con las distancias en línea recta de cada nodo de la cola hasta el nodo final. Para ello hemos utilizado la función `sort` de python o bien `np.argsort` (de numpy). Una lista ordenada no es la estructura de datos más eficiente para esta tarea, pues realmente no necesitamos contar con los datos ordenados, sino extraer rápidamente el nodo con la mínima distancia en cada iteración. En concreto, ocurre que:

- En una lista ordenada, el coste computacional de extraer el elemento mínimo (o máximo) es de tipo $O(1)$.
- En una lista ordenada, el coste de introducir un nuevo elemento (en su posición ordenada, claro está) es $O(N)$. Esto es fácil de entender, pues es necesario recorrer todo el array y colocar el nuevo elemento entre los índices $i-1$ e $i+1$.
- El coste de ordenar una lista es de tipo $O(N \log N)$.

Si el número de nodos de la búsqueda es muy alto, es preferible utilizar una estructura de tipo Min-Heap. En concreto:

- En una Min-Heap, el coste computacional de extraer el elemento mínimo es de tipo $O(logN)$.
- En una Min-Heap ordenada, el coste de introducir un nuevo elemento es $O(logN)$.

El ejemplo siguiente describe cómo definir y utilizar un Min-Heap en python.

```
import heapq
heap = []

heapq.heappush(heap, (12.5, "Node_A"))
heapq.heappush(heap, (4.2, "Node_B"))
```

```
heapq.heappush(heap, (8.9, "Node_C"))

# Consulta del mínim sense extreure'l -> O(1)
min_dist, node = heap[0]  # (4.2, 'Node_B')

# Extracció de l'element mínim: heappop() -> O(log N)
min_dist, node = heapq.heappop(heap)  # Extreu i retorna (4.2, 'Node_B')
```

Nota: **no es necesario hacer nada especial en este apartado**. Solamente se os proporciona esta información para que seáis conscientes de que el tiempo de cálculo es importante en cualquier algoritmo de búsqueda. Durante el resto de esta práctica **vamos a mantener una lista ordenada, pues es más sencillo de implementar y permite al estudiante entender mejor el progreso del algoritmo.**

2.5. A*

El algoritmo A^* es muy utilizado para el cálculo de rutas, pues comparado con el algoritmo Greedy anterior, con un poco más de esfuerzo es capaz de obtener rutas óptimas entre cualquier par de nodos de la red. Podemos pensar que el algoritmo A^* es similar al algoritmo de Dijkstra: simplemente se mofifica la función de coste añadiendo una parte heurística que aproxima el coste total hasta el final de la ruta.

En cada iteración del algoritmo, A^* debe determinar cuál de los caminos extender. Lo hace basándose en el coste del camino desde el nodo inicial n_s hasta el nodo n ($g(n)$) más una estimación del coste requerido para llegar hasta el nodo de destino ($h(n)$). Así pues, el algoritmo A^* selecciona el camino que minimiza la función:

$$f(n) = g(n) + h(n)$$

En esta expresión, llamaremos:

- $f(n)$: el coste total (que llamaremos *ranking distance*). Se prioriza la exploración de nodos para los que el coste total es mínimo.
- $g(n)$: distancia acumulada hasta el nodo actual. Esta es la suma de distancias parciales (entre nodos) hasta el nodo actual.
- $h(n)$: distancia en línea recta (*flying distance*), igual que en el algoritmo Greedy. En general, esta función debe estimar el coste del camino más corto desde n hasta el destino n_s.

Con todo esto, considere que se intenta llegar desde un nodo inicial n_s a un nodo final n_e. Nótese que durante una iteración intermedia del algoritmo se está evaluando el coste $f(n)$ para un nodo intermedio n. El coste $f(n)$ no es una distancia precisa desde el nodo n hasta n_e, ya que incluye la distancia $h(n)$ (que es un heurístico aproximado). Obviamente, de ser $h(n)$ la distancia exacta hastaa el final, habríamos terminado ya la búsqueda, pues elegiríamos en ese instante la distancia óptima (la mínima).

2.5.1. Descripción detallada del algoritmo

Igual que los dos algoritmos vistos antes, el A^* debe usar una cola de prioridad para poder seleccionar los nodos con coste $f(n)$ mínimo que se deben expandir. Esta cola abierta (`self.open_queue`), también se conoce como la frontera o el *open set*. Damos, a continuación, una descripción detallada del algoritmo:

- Inicializamos:
  - Haga $g(n) \forall n =$ ínf. Esto se puede conseguir con un `defaultdict` de python.
  - Inicializamos la lista abierta con el nodo inicial.
  - Calculamos h para el nodo inicial.
  - Inicializamos `node_info[start_name]={'parent': None}`
- Mientras la lista abierta contenga elementos.

- Obtenga el nodo n de la lista abierta con menor f. Saque n de la lista abierta.
- Añada este nodo n a la lista cerrada.
- Procese los vecinos de este vecino n:
  - Si el vecino i está en la lista cerrada: continúe.
  - Si el vecino i no está en la lista abierta: añádalo a la lista abierta y calcule g, h y actualice `node_info`.
  - Si además, se ha calculado una distancia g menor que la anteriormente vista: actualice g para ese nodo y `node_info`.

Hemos visto que, igual que en los algoritmos anteriores, se debe mantener una lista con el padre de cada nodo, de manera que se pueda reconstruir la ruta cuando se halla el nodo final.

Ejercicio 2.12

Se desea observar el avance del algoritmo durante diferentes iteraciones.
EJERCICIO
Modifique el código de `search_algorithms/a_star_algorithm.py` para hacer lo siguiente:

- Imprima por pantalla el nodo actual en cada iteración del algoritmo. Siga la evolución del algoritmo desde Madrid hasta Elche (llame al algoritmo desde `search_algorithms.py`.
- Imprima por pantalla los vecinos del nodo actual.

Una vez tenga implementada la clase, descomente las siguientes líneas:

```
algoritmo = A_star_Algorithm(spain_network)
origen = 'Madrid'
destino = 'Elx'
route, distance, iterations = algoritmo.find_route(origen, destino)
```

Ejercicio 2.13

Se desean probar diferentes rutas entre diferentes nodos del mapa.
EJERCICIO
Pruebe el nuevo algoritmo A^* con 5 orígenes y destinos diferentes (las mismas que ha utilizado en el ejercicio 2.10. Para cada una:

- Anote la ruta calculada.
- Anote el número de iteraciones del algoritmo.
- Anote la distancia total de la ruta calculada. ¿La ruta hallada es óptima?

Si repetimos, de nuevo, nuestro experimento Madrid-Elx, obtenemos:

```
Found destination! in iterations:  10
Route found: Madrid -> Tarancón -> Albacete -> Villena -> Alicante -> Elx
TOTAL DISTANCE IS:  443
```

El algoritmo necesita 10 iteraciones para encontrar una ruta entre Madrid y Elche, un número mayor que el algoritmo Greedy anterior. Sobre los saltos, se encuentra una ruta con 5 saltos. La ruta encontrada es la mínima entre las dos ciudades (443 km) en términos de distancia. En este caso, el algoritmo no se equivoca buscando el nodo Cuenca y elige el nodo Tarancón, pues la distancia $f(s) = g(s) + h(s)$ para Tarancón es menor. Observe que $g(s) = 85km$ para Tarancón y $h(s) = 165km$ para Cuenca.

Como en el caso del algoritmo avaro, el algoritmo A^* mantiene un número de iteraciones bajo, pues explora únicamente el nodo con distancia $f(s)$ mínima. En este caso, la distancia $f(s)$ no es simplemente la distancia en línea recta $h(s)$ del algoritmo avaro (que vimos que no siempre encontraba una ruta óptima). En el algoritmo A^* la distancia $f(s) = g(s) + h(s)$ mantiene una visión más global de la tarea, teniendo en cuenta, además, la distancia $g(s)$ necesaria para llegar al nodo i. Esto le da

al algoritmo una visión de conjunto similar a lo que haría un humano para encontrar una ruta óptima en un mapa (aunque ciertamente, nadie ha consultado un mapa en papel en los últimos 10 años).

En concreto, se puede demostrar que para que el algoritmo A^* pueda encontrar siempre una ruta óptima, la heurística $h(s)$ debe ser admisible, es decir: $h(s)$ **nunca debe sobreestimar el coste real para llegar al destino**. En el caso que nos ocupa, esto es cierto siempre, pues la distancia $h(s)$ es la distancia en línea recta entre un nodo y el destino. Desde cualquier nodo del grafo hasta el destino siempre tendremos una distancia d por carretera mayor que $h(s)$ ($h(s) < d$ y, por tanto, $h(s)$ no la sobreestima), pues viajamos por carretera y las carreteras tienen curvas.

3. Resumen

Si volvemos a probar los tres algoritmos y buscamos una ruta entre el nodo de origen y el destino, obtendremos:

- BFS:

```
Found destination! In iterations:  17
Route found: Madrid -> Cuenca -> Albacete -> Murcia -> Elche
TOTAL DISTANCE IS:  500
```

  El algoritmo necesita 17 iteraciones para encontrar una ruta entre las dos ciudades. La ruta obtenida por el algoritmo es aquella con un número de saltos mínimo (4, 4 flechas). La distancia obtenida (500 km) no es la mínima entre los nodos (pues el algoritmo no usa ninguna información de distancia).

- *Greedy*:

```
Found destination! in iterations:  6
Route found: Madrid -> Cuenca -> Albacete -> Villena -> Alicante -> Elche
TOTAL DISTANCE IS:  493
```

  El algoritmo necesita 6 iteraciones para encontrar una ruta entre las dos ciudades. Sin duda, proporcionar la distancia en línea recta como heurístico permite centrar la búsqueda en los nodos más prometedores (que se acercan al nodo de destino). La ruta entre nodos tiene 5 saltos (5 flechas), 1 más que el algoritmo BFS. El algoritmo Greedy intenta minimizar la distancia, siendo el número de saltos irrelevante. **La ruta obtenida entre las dos ciudades no es la que tiene una distancia mínima**. En concreto, en la primera iteración, el algoritmo decide de forma equivocada viajar a Cuenca, con una flying distance menor a Elche. A partir de esa equivocación el algoritmo sigue avanzando de forma ávara hacia el destino.

- A*:

```
Found destination! in iterations:  10
Route found: Madrid -> Tarancón -> Albacete -> Villena -> Alicante -> Elche
TOTAL DISTANCE IS:  443
```

  El algoritmo necesita 10 iteraciones para encontrar una ruta entre Madrid y Elche. Sobre los saltos, se encuentra una ruta con 5 saltos. La ruta encontrada es óptima (443 km).

4. Haciendo un planificador de rutas real

Estás trabajando en una empresa. Se te encomienda que hagas una planificación de rutas sobre mapas reales. Tienes los siguientes requisitos:

- Estás trabajando en una aplicación militar, así pues, no puedes depender de la API de Google o la de OpenStreet Maps, ya que es posible que no tengas cobertura de datos siempre.
- Los nodos de inicio y final se deberán introducir como coordenadas GPS de origen y destino. Estas coordenadas se definen en grados sexagesimales según la definición de la figura 5.
- Se te permite bajarte los mapas libres de Open Street Maps (libres) a través de la API de Overpass Turbo (https://overpass-turbo.eu/) y guardarlos de forma local (lamentablemente, otros proveedores de mapas no ofrecen esta información).

4.1. Fichero de partida

Se proporciona al estudiante el fichero `search_osm.py`. Se muestra, en parte, aquí:

```
if __name__ == "__main__":
    print('Cargando el mapa...')
    G = ox.graph_from_xml("maps/map0.osm")
    print('Mapa cargado.')
    # SE DEBE CREAR LA CLASE A_star_Algorithm_OSM
    algoritmo = A_star_Algorithm_OSM(G)
    gps_origen = (38.2694, -0.706661)
    gps_destino = (38.25, -0.694)
    nodo_origen = algoritmo.get_closest_node(gps_origen)
    nodo_destino = algoritmo.get_closest_node(gps_destino)
    route, distance, iterations = algoritmo.find_route(nodo_origen, nodo_destino)
    plot_route(G, route, distance, zoom_level=5)
```

En este caso, debemos modificar la clase `A_star_Algorithm_OSM` (fichero `a_star_algorithm_osm.py`). En esta última tendremos que adaptar algunos de sus métodos para trabajar con la librería `osmnx` que usamos para cargar el mapa.

4.2. Datos del mapa

OpenstreetMap es un proyecto libre que ofrece datos de mapas de todo el mundo. Existe un servidor alternativo denominado Overpass-Turbo que permite descargar los datos de OpenStreetMap de forma interactiva. Para ello, se puede acceder a https://overpass-turbo.eu/ para obtener datos de cualquier zona del mundo. En la práctica se proporcionan al estudiante los mapas `maps/map0.osm`, `maps/map1.osm` y `maps/map2.osm` (véase la figura 6) que se han descargado previamente. Si el estudiante desea guardar otro mapa para experimentar con él, es necesario acceder a https://overpass-turbo.eu/, seleccionar una zona del mapa, ejecutar una consulta, exportar datos y elegir la opción de exportar como OSM (datos en bruto de OSM). La consulta a realizar se anota, a continuación:

```
[out:xml];
(
  way["highway"]({{bbox}});
);
out body;
>;
out skel qt;
```

Si el mapa es muy grande y se desea menor detalle:

```
[out:xml];
(
  way["highway"~"^(motorway|trunk|primary|secondary)(_link)?$"]({{bbox}});
);
out body;
>;
out skel qt;
```

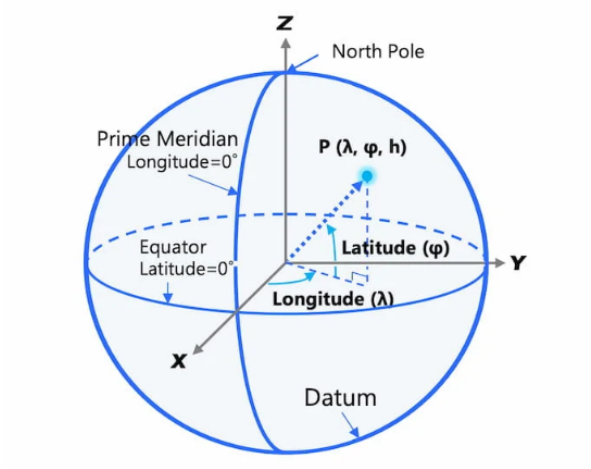


Figura 5: Coordenadas GPS.



(a) `map0.osm` (b) `map1.osm` (c) `map1.osm`

Figura 6: Tres mapas de OSM.

4.3. Adaptación del algoritmo

El algoritmo A^* que se programó utilizaba cadenas de caracteres para nombrar los nodos. Además, cada nodo representaba una ciudad de España. En el caso que estudiaremos ahora, los nodos del mapa de OpenStreetMaps son posiciones GPS de calles y carreteras. Una calle está representada por diferentes nodos. Además, OpenStreetMaps mantiene una relación de conectividad entre nodos (id est, un grafo).

Usaremos la librería `osmnx` para cargar los datos de un mapa de OSM. Esto es fácil de hacer. Con esta línea cargaremos un mapa de nodos desde un fichero de OSM.

```
G = ox.graph_from_xml("maps/map0.osm")
```

Ahora nuestro grafo es `G`. Así pues, nuestro algoritmo usará este grafo `G`. El grafo `G` mantiene una lista de los nodos que están conectados: por ejemplo, calles que se cortan. Le pasaremos el grafo `G` al algoritmo para que lo use, igual que antes usábamos el grafo de ciudades:

```
algoritmo = A_star_Algorithm_OSM(G)
```

Igual que antes, el grafo `G` se guardará en el algoritmo como `self.graph`. Con esto, para que nuestro algoritmo funcione, solamente hace falta adaptar las consultas que se hacían sobre el grafo anterior. En concreto, ahora hay que hacer las consultas sobre el grafo de tipo `osmnx`. Se anotan, a continuación, las funciones de la librería `osmnx` que necesitaremos:

- **Para saber si un nodo está en el mapa**. Ejemplo:

```
if start_name not in self.graph or destination_name not in self.graph:
    return None, None, None
```

- Si deseamos **obtener los vecinos del nodo actual**. El siguiente código devuelve los nodos que están conectados con el nodo actual:

```
self.graph.neighbors(nodo_actual)
```

- **Distancia entre dos nodos conectados** del grafo:

```
datos_aristas = self.graph.get_edge_data(current_node_name, neighbor)
rel_distance = datos_aristas[0].get('length')
```

- **Distancia *flying distance* entre dos nodos (en metros)**. Se cebe implementar usando estas funciones:

```
lat, lon = self.graph.nodes[node_name]['y'],
    self.graph.nodes[node_name]['x']
...
distance_meters = ox.distance.great_circle(lat1, lon1, lat2, lon2)
```

Ejercicio 4.1

Complete las funciones del algoritmo proporcionado para que se pueda trabajar con grafos de tipo `osmnx`.

EJERCICIO

Modifique las funciones `self.get_distance` y `self.compute_flying_distance` y `self.get_neighbors`. Estas son funciones de la clase `A_star_Algorithm` que se han dejado vacías. A su vez, estas funciones deben llamar a las funciones de `osmnx` del grafo `self.graph` (que ahora es de tipo `osmnx`). Utilice el mapa `maps/map0.osm`, pues es más rápido cargarlo y hacer pruebas sobre él.

Ejercicio 4.2

Pruebas a realizar: se anotan algunas coordenadas para trabajar con cada uno de los mapas:

- `map0.osm` (Elche).

```
gps_origen = (38.266320, -0.706661)
gps_destino = (38.166320, -0.686661)
```

- `map1.osm` (Elche-Alicante).

```
gps_origen = (38.2694, -0.706661)
gps_destino = (38.37, -0.5)
```

- `map2.osm` (Elche-Valencia).

```
gps_origen = (38.2694, -0.706661)
gps_destino = (39.48, -0.362)
```

4.4. Mejoras y tareas adicionales

Nos damos cuenta de que la carga del mapa es lenta. Esto no resulta un gran inconveniente, pues en la aplicación que se está diseñando, esta acción se realiza una vez y, a continuación, el algoritmo puede calcular y servir muchas rutas. Nos da la sensación de que el algoritmo que se ha programado funciona con bastante rapidez, sin embargo, nos gustaría poder medir este tiempo.

Ejercicio 4.3

Se desea medir el consumo del algoritmo en términos de tiempo de ejecución y memoria.
EJERCICIO
Cargue el mapa una vez y a continuación, genere 50 pares de coordenadas GPS de origen y destino aleatorias. Calcule el tiempo medio de ejecución. Intente normalizar el tiempo de ejecución respecto de la distancia de la ruta planificada por el algoritmo (rutas más largas requieren más tiempo, pues hay que explorar más nodos). El algoritmo se puede acelerar simplemente eliminando mucha de la información que se muestra por pantalla con `print()`. Busque, también, otras formas de acelerar el cálculo de las rutas.

Ejercicio 4.4

También resulta interesante que se plotee la posición GPS de todos los nodos que ha ido explorando el algoritmo. Recuerde que esta información se guarda en `self.visited_nodes`. De esta manera el estudiante obtendrá una intuición de cómo funciona este algoritmo de búsqueda.